

Report — Sanskrit↔English Semantic Retrieval (Option 2)

Numbers below are from the fine-tuning run (e5-small, MNRL, 1 epoch, Colab T4). They are run-specific; the method and analysis are fixed.

1. Problem understanding

Build a Sanskrit↔English semantic retrieval system: given a Sanskrit verse or an English query (e.g. *"what does this verse say about karma?"*), retrieve the relevant Sanskrit passage and/or English explanation. This is a **cross-lingual bi-encoder retrieval** problem — the query and the relevant document are in different languages — evaluated with ranking metrics (Recall@K, MRR, nDCG) and demonstrated in a mini-RAG pipeline. Chosen over the LLM-instruction and post-OCR tracks because it uniquely affords a clean, label-free, quantitative before/after within the compute/time budget (full scoring in 02-decision-memo.md).

2. Dataset preparation

All datasets are drawn from the assignment's suggested Option-2 list (comparison in 02-decision-memo.md):

- **Primary (training + in-domain eval):** rahular/itihasa — ~93K aligned Sanskrit↔English verse pairs (Rāmāyana + Mahābhārata), public domain. Train split → training; test split → **in-domain** eval. This is the assignment's "Sanskrit-English aligned corpora" — chosen for its volume of clean aligned pairs.
- **RAG demo + qualitative eval: Bhagavad Gita** (JDhruv14/Bhagavad-Gita_Dataset, 701 verses, sanskrit+english columns) — small, clean, iconic, and it directly serves the spec's example query.
- **Out-of-domain held-out: AI4Bharat IN22** (ai4bharat/IN22-Gen, ~1,024 n-way sentences, sentence_san_Deva/sentence_eng_Latn) — the *honest* generalization number, immune to Itihasa's archaic-epic domain skew. FLORES-200 san_Deva is the fallback; both are gated (need HF auth), so an unauthenticated run reports a clearly-labelled held-out Itihasa slice instead.
- **Deliberately skipped:** Upanishads and OPUS — sparse/uneven Sanskrit↔English alignment, low ROI in a 1–2 day budget (a Dataset-Thinking scoping call, not an oversight).
- **Pipeline** (src/.../data.py, normalize.py): Unicode **NFC** normalization → build **directional** pairs (Sa→En *and* En→Sa) with e5 query:/passage: prefixes → ****IAST** augmentation **of a fraction of the Sanskrit side (transliteration robustness)** → dedup** exact pairs → split. No synthetic positives are fabricated — alignment is real and 1:1.
- **Why 1:1 labels matter:** every query has exactly one known-relevant passage, so ranking metrics need **zero manual annotation**.

3. Why this base model

intfloat/multilingual-e5-small. Rationale and rejected alternatives:

Candidate	Decision	Reason
multilingual-e5-small	☒ chosen	XLM-R lineage → CC-100 includes Sanskrit → real coverage, weak baseline, large fine-tuning headroom; 118M/384-dim, MIT, big batches fit a T4; asymmetric query/passage prefixes fit cross-lingual retrieval
multilingual-e5-base	☐ stretch	Same lineage, higher quality; optional quality run if time permits
BAAI/bge-m3	☐	568M borderline on 16GB T4; 8192-context wasted on short verses
gte-multilingual-base	☐	From-scratch tokenizer, Sanskrit coverage unverified → correctness risk
jina-embeddings-v3	☐	CC-BY-NC (non-commercial)
IndicSBERT / MuRIL	☐	Sanskrit not among supported langs; monolingual Sanskrit MLs lack an English side

4. Fine-tuning approach

Full fine-tune (not LoRA — 118M is small enough that full FT is both feasible and better) with `MultipleNegativesRankingLoss`. MNRL uses **in-batch negatives**, so batch size *is* the negative count → we maximize batch size (T4 → 64–128). Gradient accumulation does **not** add in-batch negatives, so raw batch is prioritized; `CachedMultipleNegativesRankingLoss` provides a large effective batch under VRAM pressure (run B). fp16, warmup, 1–3 epochs.

5. Hardware constraints & optimizations

Single Colab **T4 (16GB)**. `max_seq_len=128` (verses are short) keeps activations small → larger batch → more negatives. fp16 throughout. `IndexFlatIP` (exact) for the demo/eval so ANN approximation never confounds recall. Local RTX 3050 (4GB) is dev/smoke-test only — its VRAM caps batch size, which specifically starves in-batch negatives (see 01). Bonus experiments run: INT8 quantization (100% recall retention, 4× smaller index) and an e5-base comparison — see section 6.

6. Evaluation methodology & results

`InformationRetrievalEvaluator` → Recall / MRR / nDCG @ {1,5,10}, on (a) in-domain Itihasa test and (b) an out-of-domain set. Headline = **base vs fine-tuned** on identical eval sets. Run: e5-small, MNRL, batch 64, 1 epoch, `max_seq_len 128`, fp16, 168,783 training pairs (Itihasa train + 25% IAST augmentation, deduped). Devanagari tokenizer fertility measured at **3.36 subword-tokens/word** (confirms heavy fragmentation).

In-domain (Itihasa test, 11,721 held-out; queries = Sanskrit, corpus = English):

Metric	Base	Fine-tuned	Δ
Recall@1	0.147	0.789	+0.642 (5.4×)
Recall@5	0.266	0.911	+0.645
Recall@10	0.331	0.937	+0.606
MRR@10	0.199	0.841	+0.642
nDCG@10	0.230	0.864	+0.634
MAP@100	0.208	0.843	+0.635

****Out-of-domain (Bhagavad Gita — ungated, genuinely cross-domain: philosophical dialogue vs the epic-narrative training corpus).**** The gated sets (IN22, FLORES) require HF auth; when unauthorized, the notebook falls back to the ungated Gita, which is a *real* domain shift (not a same-domain proxy). Base → fine-tuned:

Metric	Base	Fine-tuned	Δ
Recall@1	0.174	0.718	+0.544
Recall@10	0.370	0.940	+0.570
MRR@10	0.229	0.797	+0.568
nDCG@10	0.262	0.832	+0.570

Slightly below in-domain (R@1 0.718 vs 0.789) — the honest, expected cost of generalizing to a new corpus. This is the trustworthy generalization number; an earlier run that fell back to a *same-domain* Itihasa slice reported an inflated R@1 0.812, which overstated generalization and was discarded.

Bonus results:

- **e5-base vs e5-small (both fine-tuned, in-domain):** e5-base wins — Recall@1 **0.877 vs 0.789**, Recall@10 0.973 vs 0.937, nDCG@10 0.925 vs 0.864. ~+9 pts Recall@1 for ~2.4× params.
- **INT8 embedding quantization:** Recall@10 **0.9385 vs 0.9370 float32 (100.2% retention)**; index **3.1 MB → 0.8 MB (4× smaller)**. Effectively free — a strong deployment lever.

Iteration log:

#	Config	Recall@1	Recall@10	nDCG@10	Note
base	e5-small, no FT	0.147	0.331	0.230	weak Sanskrit baseline (expected)

A	e5-small, MNRL, bs64, 1ep	0.789	0.937	0.864	headline run
bonus	e5-base, MNRL, bs64, 1ep	0.877	0.973	0.925	best quality
OOD	e5-small run A, Gita cross-domain	0.718	0.940	0.832	honest generalization
B (pre-fix)	e5-small + naive hard-negs (20K)	0.676	0.884	0.778	regressed — false negatives (section 7)
B (margin)	+ margin guard 0.05 + range_min=25	0.147	0.331	0.230	guard rejected 99.6% → only 20 valid triplets → ~untrained (section 7)

7. Failure cases

- **Hard-negative mining does not fit this corpus (headline finding — a documented negative result).**

Two attempts, both instructive: (1) *naive* mining regressed vs run A (Recall@1 0.676 vs 0.789) — the mined "negatives" had **higher** mean cosine to the anchor (0.887) than the positives (0.861), i.e. the epic corpus is dense with near-duplicate shlokas semantically equivalent to the gold, so the miner selected relevant verses as negatives (false negatives). (2) Adding a strict false-negative guard (margin 0.05, range_min 25) ****rejected 99.6% of candidates → only 20 valid triplets survived → the model was effectively untrained (Recall@1 0.147 = baseline).**** Conclusion: once false negatives are correctly filtered, this corpus contains almost no valid hard negatives — so ****in-batch negatives (run A) are the right choice here.**** (The principled fix is a guide-model loss such as CachedGISTEmbedLoss, which masks false negatives *inside* the batch without separate mining.)

- **Transliteration robustness — PASS.** With a valid in-corpus test (querying the same verse in Devanagari and in IAST), the fine-tuned model retrieves the **same top-1 for both scripts** — the 25% IAST augmentation worked. (An earlier apparent mismatch was a test artifact: it queried an out-of-corpus verse, so neither script could retrieve a gold that was not indexed.)

- **Domain shift.** Itihasa's 19th-c. archaic English inflates in-domain metrics; the ungated Gita cross-domain eval (R@1 0.718 vs in-domain 0.789) exposes the real, modest generalization gap.

- **Tokenizer fragmentation.** Measured Devanagari fertility **3.36 tok/word** caps achievable quality and eats context; documented, not fixed (tokenizer surgery is out-of-scope in-budget).

8. Challenges encountered

Cross-lingual asymmetry (correct query/passage prefixing is a silent-failure landmine — pinned by a unit test), transliteration handling, honest eval framing (1:1 verse retrieval is **translation retrieval**, easier than open QA — stated, not hidden), and keeping training within T4 VRAM while preserving a large negative count.

9. What I'd improve with more time

Already explored (bonus): e5-base (higher quality), INT8 quantization (free 4× index shrink), and hard-negative mining (regressed on naive settings → fixed with a false-negative margin guard, section 7). Remaining: a true out-of-domain number via authenticated IN22/FLORES (or the wired Gita tier); a hand-labelled thematic query set for RAG-realistic eval beyond 1:1 translation retrieval; the reverse (En→Sa) retrieval direction reported alongside Sa→En; a query-time script-normalization step to close the transliteration gap; more epochs / larger batch on e5-base; embedding-space UMAP before/after visualization; and the production hardening designed in 05-production-system-design.md.